HumanTrace2Workflow&Verifier

Compiling a Person's Workflows and Their Verifiers from the Access Records They Already Have

Zhi Zheng

National University of Singapore

Working draft. Results are preliminary: one run per configuration, synthetic samples.

Abstract

People repeat many procedures on a computer, yet handing one of them to an agent still means describing it, prompting for it, and checking the result by hand. Recent products turn a single recorded demonstration into a reusable skill, and recent work induces task models from passive traces. Both recover *how* a task is done. Neither recovers what the person accepts as *done*. We argue that the two are inseparable: there is no best workflow, only a best workflow under a particular person's acceptance standard, so the object to learn is a pair (workflow, verifier). The evidence for the verifier does not lie on the main line of a task but beside it, in corrections, interruptions, re-checks and rework, and much of it is already written down in the logs of the coding agents a person uses. HT2WV collects such access records with per-source consent, compresses them deterministically into a timeline with off-mainline signal candidates, and opens headless sessions in an existing coding agent to distil a profile and, per task, a workflow delivered as a sequence of session prompts plus a layered verifier that every session receives as one shared prompt. To measure this we build synthetic samples in which a pre-filled personal style conditions the records and is then hidden: the system is scored on how much of the person's rubric its workflow satisfies and on how many hidden style items its verifier catches, including a seven-day, stateful sample in which doing the right things in the wrong order changes the outcome. In first runs, single-step tasks are saturated at the first attempt, while half of the learned verifiers reject the output the person would have kept: coverage without acceptance of the reference is not a verifier.

1 Introduction

A computer-use agent can operate an interface, but *what* it should do and *how good* the result must be are still specified by hand. Two lines of work reduce the first burden. Record-and-replay features in current agent products turn one narrated demonstration into a skill [1, 2, 3], and task model induction recovers goal hierarchies and procedures from passively recorded traces [4]. Both produce a description of the procedure. The second burden, deciding whether a result is acceptable, stays with the person.

That standard is personal. Two people writing the same weekly report may differ in exactly what makes it acceptable: one sends nothing until every figure has been checked against the source file and the file is named at the bottom; the other wants a one-paragraph takeaway in a friendly tone. An agent optimised for one of them is wrong for the other (Figure 1a). We take this literally:

> *Workflow and verification are coupled. The thing to learn is not a procedure but a pair (workflow, verifier), and the verifier belongs to a person.*

Three consequences follow. Personalisation is part of the problem statement, not a feature. The verifier is the durable asset: when a stronger model arrives the workflow can be regenerated, possibly as a completely different procedure, while what the person accepts changes slowly. And optimisation becomes well defined: hold the person's verifier fixed and search over workflows.

Where the verifier's evidence is. It is not on the main line of the task. It is in what task mining discards as noise: undoing and redoing, glancing back at a source without copying anything, hesitating before

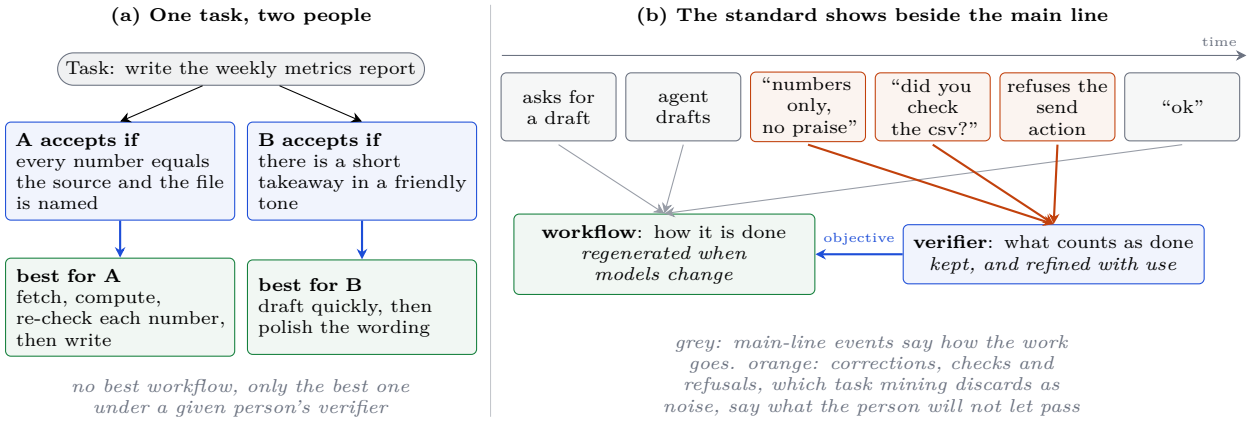


Figure 1: The idea. (a) The acceptance standard differs between people, and it decides which workflow is best. (b) Evidence for that standard lies off the main line of the task, and is densest in the person's existing conversations with coding agents.

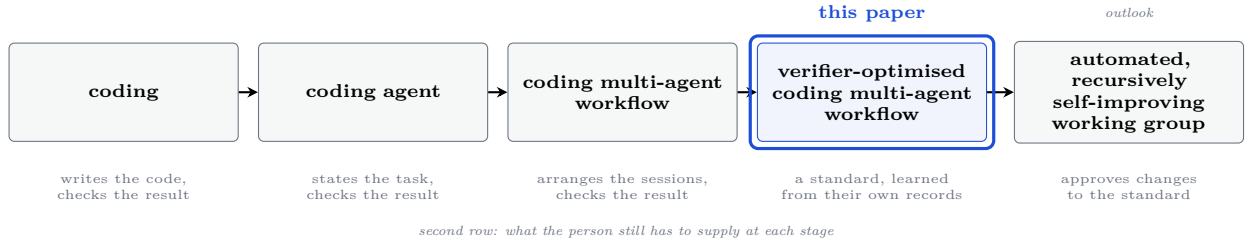


Figure 2: The trajectory we have in mind. Each stage removes something the person had to supply. This work is the fourth stage: the multi-session workflow is optimised against a verifier learned from the person's records.

sending, and, most densely, in how the person talks to their coding agents. Every "that's wrong", "did you check it?", interrupted turn and refused tool call is an acceptance criterion stated in the person's own words, time-stamped, with the rejected output one message above it (Figure 1b). These logs already exist on the machine; nothing has to be recorded.

Where this sits. We see the automation of work with coding models as five stages (Figure 2): *coding*, where a person writes with a model's help; the *coding agent*, where one agent carries a task from start to finish; the *coding multi-agent workflow*, where several agent sessions are arranged into a procedure; the *verifier-optimised coding multi-agent workflow*, where that arrangement is no longer made by hand but searched for under an explicit acceptance standard; and the *automated, recursively self-improving working group*, a standing group of agents that maintains and improves its own workflows over time. Each stage takes away one thing the person had to supply: the keystrokes, the step-by-step supervision, the arrangement of the steps, and finally the upkeep of the arrangement. This paper implements the fourth stage. The object that makes it possible is the same one that would make the fifth safe to attempt: a verifier that is explicit, executable and owned by the person, so that what is being improved (the workflow) and what judges it (the verifier) are kept apart, and a group that rewrites its own procedures still answers to a standard it cannot rewrite on its own.

Contributions.

- A formulation of personal automation as learning a pair (workflow, verifier), with a verifier in five layers (objective, process, style, preference, report) plus a checking policy (§4).
- HT2WV, an open implementation that reads access records a person already has under per-source consent, calls no model API itself, and delegates all reading and writing to headless sessions of an existing coding agent whose outputs it constrains with deterministic checks.

- A sample format and scoring protocol with a hidden, pre-filled personal style, including an order-sensitive, seven-day stateful task, and first measurements on six samples (§5–6).

2 Problem

A person u leaves *access records* R_u: conversations with coding agents, shell history, browser history, exported chats, file names and modification times, optionally screen recordings. For a recurring task the person has a *purpose* and can state a short *rubric* Q_u, the items they would tick off. Behind it is a larger *style* S_u: standards they apply but rarely state, about results, about the order in which things are done, and about how results are reported back to them.

From R_u and Q_u the system produces

- a *workflow* W: an ordered list of prompts, one per agent session; sessions share no memory and hand over through files;
- a *verifier* V: a set of items, each a claim with either an executable check or a judging rubric, a layer, a weight, and pointers to the records that justify it; and a *policy* describing how this person checks (how deep, when, what to do on failure, when to stop and ask).

V is rendered into a single prompt that every session of W receives, so the procedure may be cut into any number of sessions but there is one definition of done.

We measure three things on a held-out instance of the task with reference output $y^\star$ (what u would keep):

$$\begin{aligned} \textbf{rubric completion}: &\quad |\{q \in Q_u : q(W(x)) \text{ holds}\}| \,/\, |Q_u|, \\ \textbf{style coverage}: &\quad |\{s \in S_u : V \text{ fails } y^\star_{\neg s} \text{ on an item it passes on } y^\star\}| \,/\, |S_u|, \\ \textbf{reference acceptance}: &\quad V(y^\star) = \text{accept}, \end{aligned}$$

where $y^\star_{\neg s}$ is the reference broken in exactly the way s forbids. Coverage rewards a strict verifier; reference acceptance penalises one that is strict about the wrong things. For long-horizon tasks rubric completion is reported per day, and the headline number is how many days are *filled*, i.e. pass every applicable rubric item.

3 An example: a week of refunds

Figure 3 shows our hardest sample. Xu Qing handles after-sale refunds for a small shop. Each day she receives a sheet of refund requests and works through them in a back office where every action is real: a refund cannot be undone, and neither can a notification to the buyer. The back office allows at most ¥1000 of refunds per day and declines anything beyond that.

What the system is given is short: the purpose, a rubric of seven items (results equal hers; every order is looked up before anything is done to it; the buyer is notified once, last; nothing declined or refunded twice; the report's first line states the day's totals; on Sunday a week summary), and her access records: about sixty agent turns, twenty-three shell commands, twenty-three chat messages and five page visits, roughly half of them unrelated.

What it is not given are the seventeen standards from which those records were written. Six govern order and gating within a day (oldest request first, not sheet order; a buyer-caused return loses the shipping fee; above ¥500 escalate; refund before notifying; no refund until the return is signed for; never split a refund to fit the cap). Five govern how she wants to be told (verdict in the first line; one row per order, no narration; what needs her at the top, marked; amounts as ¥ with two decimals; no logs). Six only exist across days (yesterday's unpaid orders join today's queue by their original request time; an approved order is refunded the day it is approved; a re-request after a completed return is not a duplicate; a re-request after a refund is; a rejection is told to the buyer; the week summary follows the same reporting rule).

The task is built so that a correct-looking workflow in the wrong order fails. The bottom of Figure 3 shows Monday. Handled by request time, four orders are refunded and the latest request waits for tomorrow's cap. Handled in sheet order, every individual action is still legitimate, but a later request consumes the cap and an earlier one is turned away; the back office ends in a different state, and on Tuesday a different order is at the head of the queue. The reference week and every process probe are produced by actually running the back office, once following her rules and once per rule with that rule ignored, so that journal, state and report remain mutually consistent: a verifier can only catch a probe by knowing the rule.

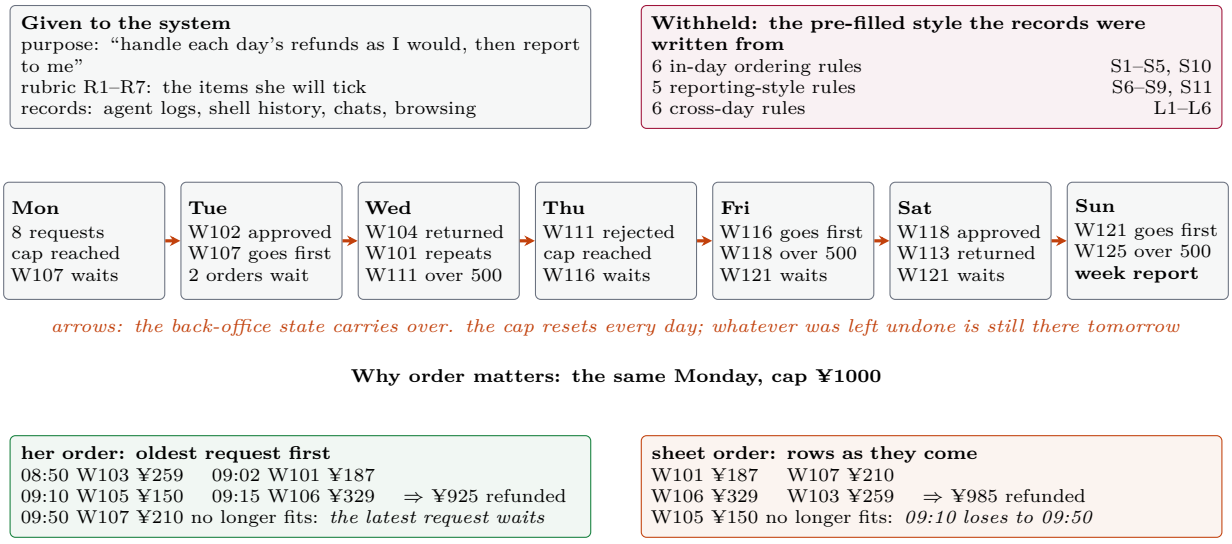


Figure 3: The seven-day sample. Top: what the system sees and what is withheld. Middle: the week; orders that could not be paid, approvals and returned parcels connect the days. Bottom: the same Monday handled in two orders.

4 Method

Collect, with consent. Each source is discovered without being opened and is read only after the person allows it: local and remote coding-agent logs, shell history, browser history (host and path only), exported chats, recently opened files, git commits. Screen recording and video import are optional extras. All text is redacted before storage. Every source writes the same event format.

Compile, without a model. A day of events becomes a timeline and a list of *off-mainline signal candidates*. Conversation signals need only logs: a correction after an agent reply, a request for proof, an interrupted turn, a refused tool call, a user-authored task specification. Screen signals need recording: a glance at another window that copies nothing back (a cross-check, as opposed to a look-up that does), undo bursts, a file saved again after the person had moved on (which yields a rejected and a kept version of the same artefact without any labelling). Detectors are deliberately high-recall; each hit is a pointer for the agent to interpret, not a conclusion.

Distil, in someone else's agent. HT2WV calls no model API. It writes a task brief (outputs, hard constraints, stop conditions, self-check) and opens a headless session in an installed coding agent. One session writes a profile, the person's working habits, acceptance style, reporting style and ordering habits, each with evidence, and lists candidate tasks. One session per task then writes the skill.

The pair. The workflow is a sequence of stage prompts. The verifier has five layers: *objective* (person-independent correctness), *process* (order and gating of actions, checked on the journal an execution leaves), *style* (this person, across tasks), *preference* (this person, this task) and *report* (structure, length and readability of what is reported back). Every personal item must cite records or be marked a low-confidence hypothesis. A lint step rejects a verifier that fails outputs the person kept, or that has nothing able to fail outputs the person discarded.

Execute and revise. Each stage runs as its own session; the verifier prompt is appended to all of them, and the last stage runs the executable checks. After a run the person's rubric verdicts go to an *iterate* session that may repair stages and add evidence-backed verifier items. In the optimisation loop proper the verifier

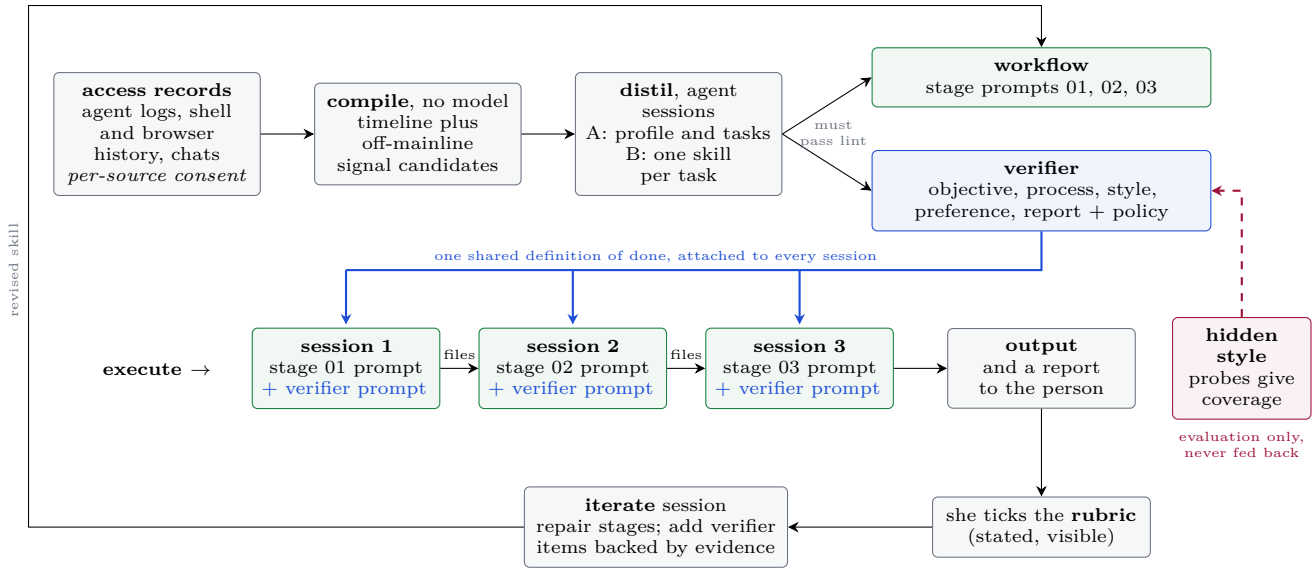


Figure 4: The pipeline. Records are collected under consent and compressed without a model; agent sessions distil a profile and, per task, a workflow and a verifier; the workflow runs as several sessions that all receive the verifier as the same prompt. Rubric failures drive revisions. Hidden style items are used only for scoring.

is frozen by hash while the workflow is revised, the reviser never sees held-out results, and a change to the verifier takes effect only after the person approves it.

5 Samples with a hidden style

A sample is built in the order a person exists: first the style, then the behaviour. We write a persona, a purpose, a rubric and a list of style items; we then write access records in which each style item leaves the kind of trace it would naturally leave (a pushed-back draft, a question about whether something was checked, a remark to a colleague, a refused action), mixed with unrelated activity. Records are rendered into each source's native format (agent session logs, time-stamped shell history, a browser history database, a chat export) and read back by the real collectors. The style list and the reference output are withheld. A validity check requires that every style item conditions at least one record and that the reference passes the sample's own rubric. Table 1 lists the six samples.

sample	person	task	record sources	hidden items	horizon
weekly-report	growth analyst	two csv files → report	2 agents, shell, web, chat	7	1 step
swe-bugfix	backend engineer	issue → fix, test, commit msg	agent, shell, web, chat	7	1 step
guest-reply	guesthouse owner	guest message → reply	agent, web, chat	7	1 step
video-publish	cooking vlogger	shoot notes → publish metadata	agent, web, chat	7	1 step
refund-ops	after-sale operator	one day of refunds, stateful	agent, shell, web, chat	11	1 day
refund-week	after-sale operator	seven days of refunds, stateful	agent, shell, web, chat	17	7 days

Table 1: Samples. In the two refund samples the order of actions changes the final state.

6 First results

Setup. All sessions are Codex CLI sessions with `gpt-6-astra` at `xhigh` reasoning effort on one server. Each sample was run once. For the single-step samples we run the built skill, give the rubric verdicts to one iterate session, and run again; judged verifier items are evaluated only in the last round.

	rubric completion		style coverage		reference
sample	round 0	round 1	checks only	with judged items	accepted
weekly-report	5/5	5/5†	6/6	6/6	yes
swe-bugfix	4/5	5/5	1/6	6/6	**no**
guest-reply	5/5	4/5	4/6	6/6	**no**
video-publish	5/5	5/5	5/6	6/6	yes

Table 2: Single-step samples, one run each. One hidden item per sample is a process standard without an output probe and is not scored. †Originally scored 2/5 because one of our rubric items wrongly required list bullets; the value shown is the same output re-scored after correcting the rubric.

Single-step tasks are too easy. Three of four workflows satisfy the whole rubric on the first execution, and all hidden items with a probe are caught once judged items are evaluated (Table 2). In these samples the output is one static file, the order of work does not matter, and each standard was stated almost verbatim by the person in a conversation. They validate the plumbing and little else, which is why the refund samples were built.

Coverage alone is misleading. For two of the four samples the verifier produced after one revision *rejects the reference output*, the reply or patch the person would have kept. In *guest-reply* the revised verifier fails the reference on three hard items: that the stated early check-in fee omits the condition "subject to availability", that the quoted total does not include the pet fee, and that the output carries no execution record proving the reply was not sent. Each is defensible in isolation and none is hers. The same revision made the workflow so cautious that it wrote a block notice next to a correct reply and lost a rubric item it had satisfied before. A verifier that fails everything would score full coverage; reference acceptance has to be reported with it, and the revision step needs the person's kept outputs as a regression set.

Scoring artefacts are easy to introduce. The apparent drop of *weekly-report* from 5/5 to 2/5 was ours: the rubric demanded list bullets the person never asked for. We keep the incident in the table because it is the same failure as the previous paragraph on the benchmark's side, an over-specific check standing in for a standard.

The week. [refund-week result: days filled out of 7, per-day rubric matrix, style coverage by group (in-day order / reporting / cross-day), reference acceptance per day. Run in progress at the time of writing; fill in from `bench-report.md`.]

Caveats. These numbers are from one run, one model and samples whose records we wrote ourselves to express the style, so they are closer to an upper bound than to an estimate. There are no human participants yet. The system has not been run end to end on a real person's records with an agent backend; on the author's machine only the deterministic stages have been exercised on real logs.

7 Related work

Demonstration to skill. Products now turn one recorded, sometimes narrated, demonstration into a reusable skill [1, 2, 3]. They are single-shot, need the person to perform, and record the main line. *Passive traces to task and user models.* Task model induction [4] separates interleaved tasks in unlabeled traces and builds goal hierarchies with control flow; general user models [5] maintain confidence-weighted propositions about a user from screenshots. Neither produces acceptance criteria. A study of mining skill files from GUI trajectories reports readable clusters with little downstream benefit [6], which we read as the absence of an objective. *Workflows from agent trajectories* [7] take success from a benchmark evaluator. *Skills with verifiers.* EvoSkills [8] co-evolves a skill generator and a surrogate verifier and reports large losses without the verifier; agentic rubrics [9] have an agent write a weighted checklist from a repository. Their verifiers are derived from the task, not from a person. *Preferences from implicit feedback.* PRELUDE [10] infers latent preferences

from a user's edits to agent output and uses them to condition generation. We extend the signal from edits to corrections, refusals and ordering habits, and use the result as an executable acceptance test. [This section relies on abstracts and secondary summaries; verify each claim against the papers before submission.]

8 Limitations and next steps

The central claim, that a workflow optimised under another person's verifier is accepted less often, is not yet tested; it needs two people (or two styles over the same task) and a cross-evaluation. The synthetic records are kinder than real ones: the person states most standards outright, and little contradicts them. Harder samples should include standards that are only ever shown, never said; standards that change over time; and colleagues whose different habits appear in the same chats (the refund samples contain a few). Isolation of held-out material relies on working directories, not on an operating-system sandbox. Finally, the verifier is only as good as the person's own blind spots; the objective layer and the visibility of every item are the current answer to that.

References

[1] Anthropic. Record a Skill in Claude Cowork. Product feature, 2026.

[2] OpenAI. Record & Replay in the Codex desktop app. Product feature, 2026.

[3] Microsoft. Skill Recorder. https://github.com/microsoft/skill-recorder, 2026.

[4] Y. Jiang, Z. Z. Wang, R. Chen, D. Yang. Inducing Task Models from Computer-Use Traces. arXiv:2608.20319, 2026.

[5] O. Shaikh et al. Creating General User Models from Computer Use. arXiv:2505.10831, 2025.

[6] Mining skill files from GUI trajectories. arXiv:2606.20363, 2026. [authors, exact title]

[7] Z. Z. Wang et al. Agent Workflow Memory. arXiv:2409.07429, 2024.

[8] EvoSkills: co-evolving skills and surrogate verifiers. arXiv:2604.01687, 2026. [authors, exact title]

[9] Agentic Rubrics as Contextual Verifiers for SWE Agents. arXiv:2601.04171, 2026. [authors]

[10] G. Gao, A. Taymanov, E. Salinas, P. Mineiro, D. Misra. Aligning LLM Agents by Learning Latent Preference from User Edits. NeurIPS, 2024.